AI-ASSISTED CODING

ASSISGNMENT 8.5

NAME:V.SIDDARTHA
HALL:2303A52062
BATCH:45

Task Description #1 (Username Validator – Apply AI in Authentication Context)
• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username(username) and then implement the function using Test-Driven Development principles.
• Requirements:
o Username length must be between 5 and 15 characters.
o Must contain only alphabets and digits.
o Must not start with a digit.
o No spaces allowed.
Example Assert Test Cases:
assert is_valid_username("User123") == True
assert is_valid_username("12User") == False
assert is_valid_username("Us er") == False
Expected Output #1:
• Username validation logic successfully passing all AI-generated test cases.

PROMPT:
Generate at least 3 assert test cases for is_valid_username(username), then implement the function using TDD principles.

Rules:
1. Length must be between 5 and 15.
2. Only alphabets and digits allowed.
3. Must not start with a digit.
4. No spaces allowed.
CODE:

```
def is_valid_username(username):
    if not isinstance(username, str):
        return False

    if not (5 <= len(username) <= 15):
        return False
```

```
    if username[0].isdigit():
        return False

    for ch in username:
        if not ch.isalnum():
            return False

    return True

# AI-generated assert test cases
assert is_valid_username("User123") is True
assert is_valid_username("12User") is False
assert is_valid_username("Us er") is False
assert is_valid_username("Ab1") is False
assert is_valid_username("ValidUser2026") is True

print("Task 1 passed: Username validation logic works correctly.")
```

OUTPUT:

ask Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

• Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.

• Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"

assert classify_value(7) == "Odd"

assert classify_value("abc") == "Invalid Input"

Expected Output #2:

• Function correctly classifying values and passing all test cases.

PROMPT:

Generate at least 3 assert test cases for classify_value(x), then implement it using conditional logic and loops.

Rules:

1. If input is integer, classify as Even or Odd.
2. If input is 0, return Zero.
3. If input is non-numeric, return Invalid Input

CODE:

```
def classify_value(x):
    if not isinstance(x, int):
        return "Invalid Input"

    if x == 0:
        return "Zero"

    # Loop-based parity check using repeated subtraction by 2
    n = abs(x)
    while n >= 2:
        n -= 2

    if n == 0:
        return "Even"
    return "Odd"

# AI-generated assert test cases
assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid Input"
assert classify_value(0) == "Zero"
assert classify_value(-10) == "Even"

print("Task 2 passed: Value classification works correctly.")
```

OUTPUT:

```
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/8.5 ASSIGN.PY/TASK1.PY"
Task 1 passed: Username validation logic works correctly.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/8.5 ASSIGN.PY/TASK2.PY"
Task 2 passed: Value classification works correctly.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

• Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") == True

assert is_palindrome("Python") == False

Expected Output #3:

• Function correctly identifying palindromes and passing all AI-generated tests.

PROMPT

Generate at least 3 assert test cases for is_palindrome(text), then implement it by normalizing strings.

Rules:

1. Ignore case, spaces, and punctuation.

2. Handle edge cases: empty string and single character.

CODE:

```
def is_palindrome(text):
    if text is None:
        return False


    cleaned = []
```

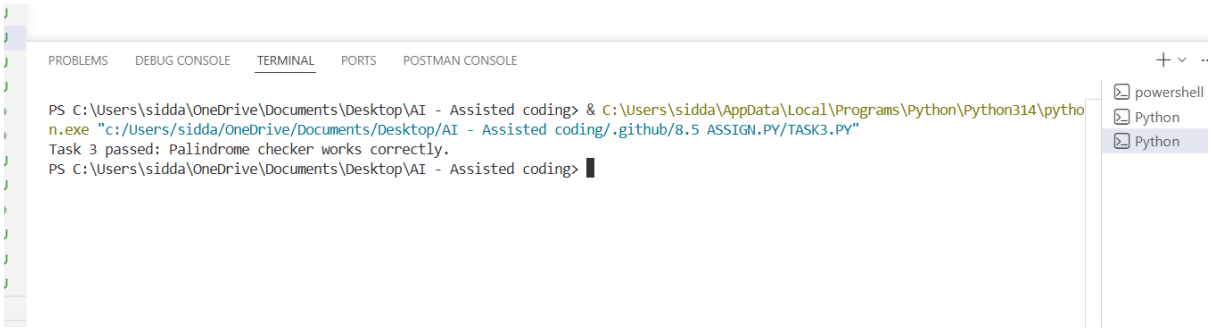
```
    for ch in str(text).lower():
        if ch.isalnum():
            cleaned.append(ch)

    normalized = "".join(cleaned)
    return normalized == normalized[::-1]

# AI-generated assert test cases
assert is_palindrome("Madam") is True
assert is_palindrome("A man a plan a canal Panama") is True
assert is_palindrome("Python") is False
assert is_palindrome("") is True
assert is_palindrome("x") is True

print("Task 3 passed: Palindrome checker works correctly.")
```

OUTPUT:



Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- **Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.**
- **Methods:**

o deposit(amount)

o withdraw(amount)

o get_balance()

Example Assert Test Cases:

```
acc = BankAccount(1000)
acc.deposit(500)
assert acc.get_balance() == 1500
acc.withdraw(300)
assert acc.get_balance() == 1200
```

Expected Output #4:

• Fully functional class that passes all AI-generated assertions.

PROMPT:

Generate at least 3 assert-based test cases for a BankAccount class with methods deposit(amount), withdraw(amount), and get_balance().
Then implement the class using TDD principles.

CODE:

```
class BankAccount:
    def __init__(self, opening_balance=0):
        if opening_balance < 0:
            raise ValueError("Opening balance cannot be negative")
        self._balance = opening_balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self._balance += amount
```

```
    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdrawal amount must be positive")
        if amount > self._balance:
            raise ValueError("Insufficient balance")
        self._balance -= amount

    def get_balance(self):
        return self._balance

# AI-generated assert test cases
acc = BankAccount(1000)
acc.deposit(500)
assert acc.get_balance() == 1500
acc.withdraw(300)
assert acc.get_balance() == 1200

acc2 = BankAccount(0)
acc2.deposit(200)
assert acc2.get_balance() == 200

# Graceful invalid operation checks
try:
    acc2.withdraw(500)
except ValueError:
    pass
else:
```

raise AssertionError("Expected ValueError for insufficient balance")

print("Task 4 passed: BankAccount class works correctly.")

OUTPUT:

Task Description #5 (Email ID Validation – Apply AI for Data Validation)

- **Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.**
- **Requirements:**

o Must contain @ and .

o Must not start or end with special characters.

o Should handle invalid formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False

assert validate_email("@gmail.com") == False

Expected Output #5:

- **Email validation function passing all AI-generated test cases and handling edge cases correctly.**

PROPMT:

Generate at least 3 assert test cases for validate_email(email), then implement

it using robust validation logic.

Rules:

1. Must contain @ and .

2. Must not start or end with special characters.

3. Should handle invalid formats gracefully.

CODE:

```
def validate_email(email):
    if not isinstance(email, str) or not email:
        return False

    allowed_start_end =
set("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789")
    if email[0] not in allowed_start_end or email[-1] not in allowed_start_end:
        return False

    if "@" not in email or "." not in email:
        return False

    if email.count("@") != 1:
        return False

    local, domain = email.split("@")
    if not local or not domain:
        return False

    if "." not in domain:
        return False
```

```
    if ".." in email or " " in email:
        return False

    domain_parts = domain.split(".")
    if any(part == "" for part in domain_parts):
        return False

    return True

# AI-generated assert test cases
assert validate_email("user@example.com") is True
assert validate_email("userexample.com") is False
assert validate_email("@gmail.com") is False
assert validate_email("user@domain") is False
assert validate_email("user..name@example.com") is False

print("Task 5 passed: Email validation works correctly.")
```

OUTPUT:

```
Task 4 passed: BankAccount class works correctly.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\Local\Programs\Python\Python314\pytho
n.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Assisted coding/.github/8.5 ASSIGN.PY/TASK5.PY"
Task 5 passed: Email validation works correctly.
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```